Use of AI assistants. Problems 7.1 to 7.4 are No AI. Problem 7.5 marks **each of its parts separately**, under the rules printed with it. Answers go in **one Haskell file**, built from the template: code as code, traces and explanations in comments. At a defense you may be asked to write or trace any function from this sheet on paper.

Two rules apply to every problem. **No partial functions**: a function without an equation for some shape of its input, or a call to `head`, `tail` or `!!`, loses the working points of its part. **The file loads**: a part you have not finished stays `undefined`, or in a comment.

Problem 7.1. No AI Consider three definitions, given without signatures:

```
countOf x xs = length (filter (== x) xs)
tag f xs     = zip xs (map f xs)
longest xs   = foldl' max 0 xs
```

(a) Say whether GHC accepts each signature below, and name what the body asks of the type.

```
countOf :: (Int -> Int) -> [Int -> Int] -> Int
countOf :: Eq a => a -> [a] -> Integer
tag     :: (String -> Int) -> [String] -> [(Int, String)]
tag     :: (a -> a) -> [a] -> [(a, a)]
longest :: [Int] -> Int
longest :: [Bool] -> Bool
```

(b) Consider a type with three values:

```
data Size = Small | Medium | Large
```

How many functions are there of type `(Size, Size) -> Size`? Write only those that GHC also accepts at `(a, a) -> a`, and explain why GHC refuses the others. Then write every function of each type below that always returns a value, or say why there is none.

```
a -> b -> a        a -> Maybe a        Maybe a -> a        (a -> b) -> a -> b
```

(c) Each signature below has a missing, redundant, or unused constraint, or none of these. Say which, whether GHC accepts the definition as written, and give the corrected signature.

```
withProgramme :: a -> [a] -> [a]
withProgramme p ms = filter (\m -> m == p) ms

busiest :: (Eq a, Ord a) => a -> [a] -> a
busiest start xs = foldl' max start xs

scale :: a -> [a] -> [a]
scale k xs = map (* k) xs

labels :: Show a => [a] -> [String]
labels xs = map show xs

within :: (Ord a, Num a) => a -> a -> a -> Bool
within lo hi x = lo <= x && x <= hi
```

Problem 7.2. No AI A washing machine reports its power draw once a minute, in watts. A cycle starts when the draw rises above zero after a reading of zero, or at the first reading if that is above zero. Consider `cycles`, which counts them with an accumulator and carries a second value, whether the machine is running:

```haskell
cycles :: [Int] -> Int
cycles ws = go 0 False ws
  where
    go n _       []       = n
    go n running (x : xs)
      | x > 0 && not running = go (n + 1) True xs
      | otherwise            = go n (x > 0) xs
-- cycles [0, 300, 0, 0, 450, 500, 0, 200] is 3, and cycles [] is 0
```

(a) Rewrite `cycles` with a left fold and no explicit recursion. The accumulator carries both values. The step may be a lambda, which can take a pattern as an equation does, or a named function in a **`where`**.

(b) Trace your version on `[0, 300, 0]`, one line per element, as the lecture traced its left fold, and say for each line which guard of `go` it corresponds to.

Problem 7.3. No AI Consider the record that the laundromat's first program used for a machine:

```haskell
data Machine = Machine
  { busy      :: Bool
  , steps     :: [String]   -- "wash 40", "rinse", "spin 1200"
  , current   :: Int        -- the index of the running step
  , outOfOrder :: Bool
  , faults    :: [String] } -- "no water", "door", "E17"
```

A machine is free, running a programme, or out of order with at least one fault. A programme is a sequence of steps, and a step is a wash (cold, warm or hot), a rinse, or a spin at some number of revolutions a minute. A running machine is on one step and has some steps left. A fault is a lack of water, a jammed door, or an error code.

(a) Write three sample values of type **`Machine`** that describe no real machine, with a line for each on what is wrong.

(b) Declare types in which none of your three values can be written, and in which every machine described above can. Call the type of a machine's state **`State`**, and the type of a fault **`Fault`**. Name the slogan of lecture 7 that this follows, and the two flaws named in lecture 6 that the old record shows.

(c) Write `advance :: State -> State`, for the moment the current step finishes: the machine goes on to the next step, or becomes free after the last one. A machine that is free or out of order stays as it is.

(d) Write `faultReport`, which takes the faults of a machine that is out of order and gives the line for its display: `"door jammed"` for one fault, and `"door jammed (and 2 more)"` for three, with the first fault named.

Problem 7.4. No AI Consider the firmware version of a machine, a list of numbers. The version `2.1` and the version `2.1.0` are the same:

```haskell
newtype Version = Version [Int]
```

(a) Write an `Eq` instance that gives

```haskell
Version [2, 1] == Version [2, 1, 0]   -- True
Version [2, 1] == Version [2, 1, 1]   -- False
```

(b) Argue in one or two sentences that your `==` is symmetric: `x == y` gives the same answer as `y == x`, for all versions.

(c) Argue in one or two sentences that it is transitive: if `x == y` and `y == z`, then `x == z`.

Problem 7.5. Consider the laundromat's booking API, documented in words and by examples:

> Each reservation has an id and a customer. It is *pending*, with the requested time; *confirmed*, with a machine and one or more consecutive time slots; *cancelled*, either by the staff, who always give a reason, or by the customer, who is refunded in full when cancelling at least two hours ahead and not at all otherwise; or *completed*, with the machine and the length of the cycle in minutes.

```
{"id": 7, "customer": "anna", "status": "pending", "requested": "2026-10-14T18:00"}
{"id": 8, "customer": "boris", "status": "confirmed", "machine": "W2",
 "slots": ["2026-10-14T19:00", "2026-10-14T19:30"]}
{"id": 9, "customer": "anna", "status": "cancelled", "cancelledBy": "staff", "reason": "machine out of order"}
{"id": 10, "customer": "dmitry", "status": "completed", "machine": "W1", "minutes": 52}
{"id": 11, "customer": "boris", "status": "cancelled", "cancelledBy": "customer", "refunded": true}
{"id": 12, "customer": "elena", "status": "cancelled", "cancelledBy": "customer", "refunded": false}
```

(a) AI allowed Give an assistant the description, the examples, and this request:

> Model this data in Haskell, as plain data types without any JSON library. Then write `machineLoad`, the total minutes of the completed cycles on a given machine, and `staffReasons`, the reasons of all the cancellations by the staff.

Use a **basic, free** model, and record its answer in the answer file.

(b) No AI Mark, next to the recorded answer, every value its types allow that describes no real reservation, with an example value for each. Then mark what else in the answer is not idiomatic Haskell, as lectures 6 and 7 present it, with one line of reason for each.

(c) No AI Write your own model, in which none of the values you marked in (b) can be written, and the two functions on it. Write the six examples as values of your types, and run both versions on them. Then write one value from (b) in the assistant's types, and report what its functions give on it.

Use of AI assistants. Part (a) only. If you use an assistant, submit `transcript.md`: the whole exchange, verbatim and unedited. Do **not** assume that the recorded answer is correct; (b) and (c) are in your own words, and (b) with nothing marked scores nothing. Without an assistant, model the data yourself in (a) and audit your own attempt.